

---

# Inteligencia Artificial Generativa con Grandes Modelos de Lenguaje

## Práctica de Laboratorio (PECL)

### Convocatoria Ordinaria

---

*Publicado el 02 de marzo de 2026, versión 1.0.*

## Asistente experto con agentic RAG

El objetivo de la práctica es construir un **asistente experto** sobre un dominio de conocimiento a elección del estudiante. El asistente debe ser capaz de responder preguntas sobre ese dominio utilizando **Agentic RAG**, es decir, un agente basado en un LLM que decide de forma autónoma cuándo necesita buscar información en una base de documentos y con qué consulta hacerlo.

### Elección del dominio

Cada estudiante (o pareja) debe elegir un dominio de conocimiento y recopilar un corpus de documentos sobre el mismo. Algunos ejemplos:

- Normativa académica de la universidad
- Documentación de un framework o librería (React, Django, Unity...)
- Manual de un juego de mesa o una guía de un videojuego
- Legislación sobre un tema concreto (protección de datos, tráfico...)
- Documentación interna de un proyecto personal

**El corpus debe tener suficiente volumen para que el RAG tenga sentido (al menos 20-25 páginas de texto, o equivalente).** Los documentos pueden estar en cualquier formato (PDF, texto, Markdown...), aunque habrá que convertirlos a texto de alguna forma para procesarlos. Si los documentos contienen imágenes, se pueden ignorar y trabajar solo con el texto. No obstante, integrar las imágenes en el pipeline (por ejemplo, usando modelos multimodales) se valorará positivamente.

**La práctica se estructura en tres fases incrementales.**

### Fase 1: Tokenización y embeddings

Tokenizar los documentos del corpus utilizando un tokenizador moderno (idealmente, el del modelo de lenguaje que se vaya a usar en las fases posteriores), y extraer estadísticas básicas como el número total de tokens, distribución de tokens por documento, ratio tokens/palabra y vocabulario más frecuente.

Seleccionar frases o fragmentos representativos de los documentos del dominio y generar sus embeddings utilizando un modelo de embeddings de frases (por ejemplo, *gte multilingual base*). Visualizar los embeddings en un espacio bidimensional (PCA o t-SNE) y analizar los resultados: ¿se forman clústeres coherentes con la temática de los documentos? ¿Hay fragmentos que quedan

inesperadamente cerca o lejos de otros? ¿Qué se puede deducir sobre la estructura semántica del corpus?

## Fase 2: RAG

Crear un chatbot básico que utilice la API de un LLM (por ejemplo, la de Groq) con un system prompt que defina el rol, el tono y los límites del asistente. Experimentar con distintas configuraciones y mostrar ejemplos comparativos. Probar al menos 2-3 variantes del system prompt y comparar cómo cambia el comportamiento del asistente (tono, nivel de detalle, formato de respuesta...). Justificar la elección final del system prompt y los parámetros de generación en base a los resultados observados.

Dividir los documentos del dominio en chunks, generar sus embeddings y almacenarlos en una base de datos vectorial (Qdrant, Chroma, o cualquier otra).

Comparar al menos dos estrategias o configuraciones de chunking distintas (por ejemplo, tamaño fijo vs recursivo, distintos tamaños de chunk, distintos valores de overlap). Para cada configuración, mostrar ejemplos de cómo quedan los chunks y evaluar cualitativamente la calidad del retrieval con unas cuantas queries de prueba. Justificar la elección final.

Implementar una función de búsqueda que, dada una query, recupere los chunks más relevantes del corpus, construya el prompt con el contexto y genere la respuesta mediante el LLM. Probar que funciona correctamente con varias preguntas representativas del dominio. Probar también con preguntas fuera de dominio (que no tienen respuesta en el corpus) y analizar el comportamiento: ¿qué chunks devuelve el retrieval? ¿El LLM alucina una respuesta o reconoce que no tiene información suficiente? Opcionalmente, se puede experimentar con búsqueda híbrida (dense + sparse/BM25), reranking con cross-encoder, filtrado por metadata o RAG multimodal (si el corpus contiene imágenes).

## Fase 3: Agente con RAG

Convertir la función de búsqueda de la Fase 2 en una **herramienta** (tool) que el agente pueda invocar mediante function calling. El agente debe recibir la pregunta del usuario, decidir si necesita buscar información en los documentos o puede responder directamente y, si decide buscar, formular la query de búsqueda apropiada que puede ser diferente a la pregunta original del usuario). La definición de la herramienta (nombre, descripción, parámetros) debe estar bien diseñada para que el modelo la use correctamente. Opcionalmente, se pueden añadir herramientas adicionales al agente (fecha y hora actual, calculadora, búsqueda web, consulta a una API externa relevante al dominio, etc.).

Analizar el comportamiento del agente. Mostrar ejemplos donde el agente decide buscar y ejemplos donde decide no hacerlo, y valorar si las decisiones son razonables. Experimentar con la descripción de la herramienta de búsqueda para ver cómo afecta al comportamiento del agente. Mostrar ejemplos donde el agente reformula la query de búsqueda de forma diferente a la pregunta original. Comparar las respuestas del agente con las del RAG directo de la Fase 2 usando las mismas preguntas: ¿mejora la calidad al poder decidir cuándo y cómo buscar? Probar también con conversaciones de varias preguntas encadenadas (por ejemplo, una pregunta de seguimiento como “¿y qué más dice sobre eso?”) y observar si el agente es capaz de reformular la búsqueda incorporando el contexto de la conversación.

El asistente debe tener una interfaz interactiva de chat. Se sugiere usar Streamlit, aunque también puede ser una TUI en terminal. Lo importante es que se pueda conversar con el asistente de forma interactiva.

## Condiciones de entrega

1. *La práctica se realizará opcionalmente por parejas (también puede llevarse a cabo individualmente) y deberá ser entregada antes del 8 de mayo de 2026 a las 23:59, a través de la tarea correspondiente en el Aula Virtual, mediante la subida de un archivo ZIP que contenga el código fuente y la memoria.*
2. ***La práctica deberá ser implementada en el lenguaje de programación Python***
3. *La entrega fuera del plazo indicado en el Aula Virtual supondrá una reducción en la calificación final, siendo del 25% si se entrega dentro de las 24 horas siguientes a la fecha límite, o del 50% si se entrega dentro de los dos días siguientes. La entrega más allá de esos dos días no será admitida bajo ninguna circunstancia.*
4. *Se debe entregar un único archivo ZIP que contenga:*
  - ***Código fuente:*** todo el código necesario para ejecutar el asistente, incluyendo un fichero requirements.txt o equivalente con las dependencias.
  - ***Documentos del dominio:*** el corpus utilizado.
  - ***Memoria breve*** (máximo 20 páginas, en formato PDF): documento que incluya: (1) descripción del dominio elegido y los documentos utilizados; (2) decisiones de diseño tomadas en cada fase y justificación; (3) resultados de los experimentos y respuestas a las preguntas del enunciado (visualizaciones de embeddings, comparativa de chunking, análisis del comportamiento del agente); y (4) dificultades encontradas y cómo se resolvieron. La estructura y secciones concretas de la memoria se dejan a elección del estudiante.
5. *Para aprobar, es **condición necesaria que el asistente funcione correctamente** y de acuerdo a las especificaciones indicadas en el enunciado.*
6. *Se recomienda usar APIs gratuitas como Groq o OpenRouter en lugar de modelos locales.*
7. *Si la práctica es realizada por una pareja, solo uno de los integrantes deberá subirla al Aula Virtual, indicando el nombre de ambos alumnos en la portada de la memoria.*
8. *En la portada de la memoria deberán figurar los datos siguientes:*
  - ***Grado en Ingeniería [Informática / de Computadores / de Sistemas de Información]***
  - ***Curso 2025/2026, Convocatoria Ordinaria***
  - ***DNI, Apellidos, Nombre***
9. *De cara a la defensa de la práctica (prevista para el 11 de mayo de 2026), el estudiante deberá realizar una demostración en vivo del asistente y responder a preguntas sobre las decisiones de diseño y los conceptos subyacentes (embeddings, chunking, function calling, etc.).*
10. *El uso de herramientas de IA (ChatGPT, Copilot, Claude, etc.) está permitido. No obstante, en caso de que se hayan usado, la memoria deberá incluir un apartado indicando qué*

*herramientas se utilizaron, en qué partes del desarrollo y de qué forma. El estudiante debe ser capaz de explicar y justificar todo el código entregado durante la defensa.*

- 11. No se podrá copiar ni reutilizar código de prácticas de otros compañeros. En caso de detectarse plagio, la práctica no será evaluada y tendrá una calificación de 0 Suspenso, ejecutándose los procedimientos correspondientes de acuerdo a la normativa vigente.*